Module 2 · SW Design

Fundamentals of Software Design

The two forces behind every good design — and a law that keeps them honest.

Akın Kaldıroğlu · Lead Trainer

Recap & today

From 'What' to 'How'

Last time: design is decision-making, and it lives on a continuum with architecture. Today we get concrete tools to judge those decisions.

Cohesion

Does each part do one thing well?

Coupling

How tangled are the parts with each other?

Change

Will the next requirement be cheap — or painful?

What you'll leave with

Learning Objectives

- Define cohesion and coupling — and recognize them in real code.
- Explain why 'high cohesion, low coupling' is the golden rule of design.
- See how coupling turns a small change into an expensive one.
- Apply the Law of Demeter to tame dependency chains.

Section 01

Cohesion

How well the things inside a module belong together.

The big picture

Two Forces Shape Every Design

Almost everything we call 'good design' comes down to balancing two measurable forces. Keep them in mind for the rest of the bootcamp:

Cohesion — want it high

- How focused a single module is
- One clear responsibility
- Measured within a module

Coupling — want it low

- How dependent modules are on each other
- How much one must know about another
- Measured between modules

Definition

What Is Cohesion?

Cohesion is the degree to which the elements inside a module belong together — how single-minded that module is about its job.

High cohesion = a class or function that does **one thing**, and everything in it serves that thing.

See it in code

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.cohesion`

Low vs High Cohesion

A 'God' utility that does unrelated jobs vs. focused classes with one responsibility each:

Java *One class, four reasons to change*

```
// LOW cohesion — unrelated responsibilities in one class
class Utils {
    void saveUser(User u) { /* database */ }
    void sendEmail(String to) { /* SMTP */ }
    double calcTax(Order o) { /* finance */ }
    String formatDate(Date d) { /* formatting */ }
}
```

Split into **UserRepository**, **EmailService**, **TaxCalculator** — each cohesive, each with a single reason to change.

What high cohesion looks like

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.password`

A Password Value Object

The opposite of the god-utility. One concept, one class: validation, random creation, comparison, formatting and masking for a password all live here — and nothing unrelated does.

Java *one thing per method — a Password*

```
public final class Password {   // one concept, one class
  // create & validate — one place owns the rules
  public static Password of(String raw)       { ... }
  public static boolean  isValid(String raw)  { ... }
  // generate a strong random password
  public static Password random(int length)   { ... }
  // compare safely — constant-time, no early exit
  public boolean matches(String candidate)    { ... }
  // format & mask — never leak the value
  public String masked()                      { ... }
  public String maskedShowingLast(int shown)  { ... }
}
```

Ask the class

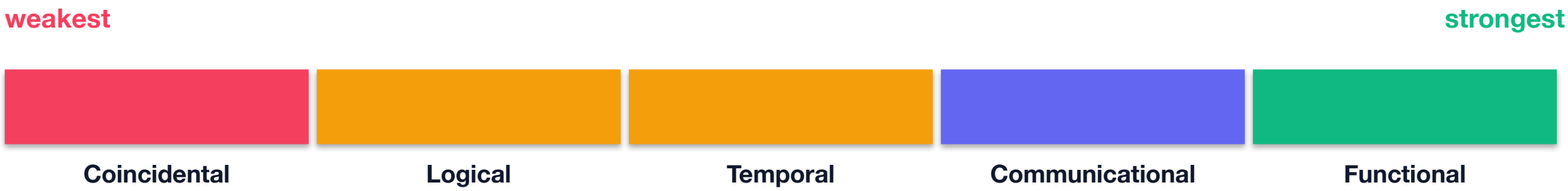
Why aren't we saving or hashing a Password here?

(Answer lives in the presenter notes.)

A useful scale

Cohesion Comes in Degrees

Not all cohesion is equal. Aim for the right end of this scale:



Functional cohesion — every element contributes to a single, well-defined task — is the goal. You don't need the jargon; you need the instinct.

Full taxonomy → `en.wikipedia.org/wiki/Cohesion_(computer_science)`

The same scale, in Java

Cohesion Degrees — the Weaker End

The weak degrees bundle code for shallow reasons: accident, category, or timing. The // comment names the smell.

Coincidental

```
class Utils {
  int add(int a, int b){}
  void log(String message){}
  boolean valid(Email email){}
}
// unrelated jobs bundled
```

Logical

```
String read(int source){
  if(source==1) file();
  if(source==2) net();
  ...
}
// same verb, a flag picks
```

Temporal

```
void startup(){
  loadConfig();
  openLog();
  connectDb();
}
// grouped by "when"
```

Rule of thumb: if the only thing the members share is a class name or a moment in time, cohesion is weak.

The same scale, in Java

Cohesion Degrees — the Stronger End

The strong degrees bind code by real, single-minded purpose: the same data, or one well-defined task. This is the goal.

Communicational

```
Stats of(Order order){
  var items = order.items();
  return new Stats(
    sum(items),
    items.size());
}
// all share one data set
```

Functional

```
double tax(double amount){
  return amount * RATE;
}
// one job, done well
```

Aim for functional cohesion: every element earns its place by serving the one job the module exists to do.

Only Functional Cohesion Scales

Every cohesion level but one absorbs new work by growing. Functional cohesion is the only one that scales: one task per unit, so new work becomes a new small unit — not more lines on an old one.

One class, growing

```
class Password {
  boolean isValid(...)  {...}
  boolean matches(...)  {...}
  String  masked()      {...}
  String  maskedLast(n) {...}
  String  redactForLog(){...} // new
  Password random(int n){...}
  Password fromPolicy(p){...} // new
}  // every feature => bigger
```

Split by task — each scales alone

```
class Password {          // a value
  boolean isValid(...) {...}
  boolean matches(...) {...}
}
class PasswordFormatter { // present it
  String masked(...) {...}
  String redactForLog(...) {...}
}
class PasswordFactory {   // build it
  Password random(int n) {...}
}
```

Growth should add new units, not new lines to old ones — split when a class does several tasks: PasswordFormatter, PasswordFactory (just as hashing became PasswordHasher).

The everyday payoff

Cohesion Shows Where Code Goes

The daily benefit of cohesion: when a new requirement lands, a highly cohesive system already tells you where it belongs.

High cohesion

- Every module has one clear job
- A new feature has an obvious home
- You add it without hunting or guessing

Low cohesion

- God classes with blurred purpose
- New code lands wherever is handy
- Cohesion decays a little more each time

A philosophy that scaled

The Unix Way: Do One Thing Well

Functional cohesion is not new — Unix has run on it for 50 years. Each tool does one thing well; pipes compose them into anything.

shell *each does one thing; the pipe composes them*

```
grep ERROR app.log |  # filter the lines
  sort |              # order them
  uniq -c |           # count duplicates
  sort -rn |          # rank by count
  head                # take the top few

# five tiny tools, one job each — composed
```

📖 **Doug McIlroy — the Unix philosophy** — *"Make each program do one thing well. To do a new job, build afresh rather than complicate old programs by adding new features."*

One job at a time

Separate Deciding from Doing

A complex condition hides a whole job inside the if. Deciding is one job, doing is another — pull the decision into a well-named method so the if reads as one clear question.

Decision buried in the if

```
if (c.age() >= 18
    && c.country().equals("TR")
    && o.total() > 100
    && !c.isBlocked()
    && c.hasVerifiedEmail()) {  // decision
  var base = o.total();
  o.setTotal(base * 1.08);      // doing
  audit.record(c, o);
}
```

Decision → its own named method

```
if (isEligibleForDiscount(c, o))  // decide
  applyDiscount(c, o);            // do
boolean isEligibleForDiscount(
    Customer c, Order o) {
  return c.age() >= 18
    && c.country().equals("TR")
    && o.total() > 100
    && !c.isBlocked()
    && c.hasVerifiedEmail();
}
```

The decision now has a name, a home, and a test — the if asks the question; the method answers it. The follow-on work is a job of its own.

Section 02

Coupling

How much modules depend on — and know about — each other.

Definition

What Is Coupling?

Coupling is the strength of the dependencies between modules — how much one module must know about another to work.

Loose coupling = modules interact through small, stable interfaces and know as **little** about each other as possible.

See it in code

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.coupling`

Tight vs Loose Coupling

Depending on a concrete class welds you to it. Depending on an interface lets you swap the other side freely:

Java — tight

```
class Report {
  MySqlDb db =
      new MySqlDb();
  // welded to MySQL
  void run() {
    db.query(...);
  }
}
```

Java — loose

```
class Report {
  final Database db;
  Report(Database db){
    this.db = db;   // any DB
  }
  void run() {
    db.query(...);
  }
}
```

The loose version depends on an abstraction — swap MySQL for Postgres or a fake in tests, no change to Report.

From Structured Design

Coupling Comes in Degrees

These classic coupling types come from Structured Design (Yourdon & Constantine) — a scale from welded-together to barely acquainted:

Aim for data coupling — modules pass only the simple values they need. Message coupling (talk through interfaces or events) is looser still.

Full taxonomy → `en.wikipedia.org/wiki/Coupling_(computer_programming)`

The same scale, in Java

Coupling Degrees — the Tighter End

Tight coupling reaches across boundaries: into another module's internals, its globals, or its control flow.

Content

```
class Report {
  void hack(Engine engine){
    engine.data[0] = 42;
  }
}
// writes Engine's field
```

Common

```
static int mode;
void writer(){ mode=1; }
void reader(){
  if(mode==1){ ... }
}
// shared global state
```

Control

```
void draw(Doc doc,
     boolean asHtml){
  if(asHtml){}
  else {}
}
// caller drives branch
```

Smell: one module can break another by touching data or logic it was never meant to see.

The same scale, in Java

Coupling Degrees — the Looser End

Loose coupling narrows the contract to exactly what is needed — ideally a few simple values.

Stamp

```
double ship(Customer customer){
  return rate(customer.zip());
}
// whole object, one field used
```

Data

```
double ship(String zip){
  return rate(zip);
}
// pass only what's used
```

Aim for data coupling: the caller hands over the minimum, so either side can change without disturbing the other.

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.coupling.oop`

Coupling in OOP: Subtyping & Message

Objects add subtyping and message. But subtyping splits in two: extends inherits an implementation (tight); implements realises an interface (loose). Messages bind the loosest of all.

Subtyping — two forms

```
// subtyping has two forms:
class Sales extends Report {}
// EXTENDS → inherits the impl (tight)
class Smtp implements Mailer {}
// IMPLEMENTS → only the contract (loose)
// = program to an interface
```

Message — loosest (best)

```
class Orders {
  final Mailer mailer;   // an interface
  void place(Order order) {
    mailer.send(order.receipt());
  }
}
// only a message crosses — nothing else
```

Program to an interface, not an implementation. — *Gang of Four, Design Patterns (1994)*

Depend on abstractions, not on concretions. — *Robert C. Martin,* **Dependency Inversion Principle** *(1996)*

The most insidious kind

Semantic Coupling

The most dangerous coupling isn't in the interface. Semantic coupling is when one module leans on another's inner behavior and assumptions — not its syntax. The compiler can't see it.

Call-order

```
parser.initialize();
parser.parse();
// parse() works only
// if initialize() ran
// — nothing enforces it
```

Magic flag

```
save(order, true);
// 'true' = skip the
// validation step;
// caller assumes what
// save() does inside
```

Partial init

```
var user = new User();
user.setName(name);
mailer.send(user);
// ok only while send()
// reads just .name —
// add a field, it breaks
```

Nothing in the syntax reveals it — so it breaks at runtime, far from the change that caused it.

📖 **Code Complete (Steve McConnell)** — *names semantic coupling the most insidious kind — it survives the compiler and breaks silently when the other module's internals change.*

Make the assumption visible

Fixing Semantic Coupling

The cure is always the same move: turn the invisible assumption into something the compiler checks — so the wrong usage stops compiling.

Call-order

```
Parser p =
  Parser.forFile(file);
p.parse();
// valid by construction —
// no init step to forget
```

Magic flag

```
save(order);
saveUnchecked(order);
// named operations —
// no hidden 'true'
```

Partial init

```
mailer.send(
  new Recipient(name,
                email));
// narrow contract —
// send()'s needs are typed
```

The assumption becomes a type or a constructor, not a comment — the compiler enforces what the caller used to have to know.

A model for the cost

Three Dimensions of Coupling

You cannot get rid of coupling — a system with none does nothing. The goal is not to remove it but to **balance** it. Khononov models its cost as three dimensions:

Integration Strength

How much knowledge two parts share.

Contract → Intrusive

e.g. a shared API contract (weak) vs. reading another module's private DB tables (intrusive).

Distance

How far apart the parts live.

Same method → Separate systems

e.g. two methods in one class (near) vs. two services owned by different teams (far).

Volatility

How often the parts change.

Stable → Volatile

e.g. the JDK's String, rock-stable, vs. a pricing rule that changes every week (volatile).

📖 **Balancing Coupling in Software Design (Khononov)** — *coupling hurts most when strength, distance and volatility are all high at once — the design job is to balance the three.*

Why it matters

Coupling Is a Change Multiplier

The cost of coupling shows up the day you change something. Tightly coupled code spreads one change across many files:

Tightly coupled

- One change ripples outward
- Hard to test a part in isolation
- Fear of touching anything

Loosely coupled

- Changes stay local
- Parts tested independently
- Confidence to evolve

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.leak`

Leaky Abstractions

Low cohesion and high coupling are one failure seen twice. A class that does not own its behavior leaks its internals — and whoever grabs those internals is now welded to them.

Leaky — internals exposed

```
class X {
  private List<Item> items;
  List<Item> getItems(){   // leaks it
    return items;
  }
}
x.getItems().add(item);     // Y reaches in
x.getItems().remove(item);  // Z reaches in
```

Sealed — behavior only

```
class X {
  private List<Item> items;
  void addItem(Item i){ items.add(i); }
  void removeItem(Item i){ items.remove(i); }
}

x.addItem(item);            // Y asks X
x.removeItem(item);         // Z asks X
```

Low cohesion IS high coupling seen from the other side — own your behavior, and expose only the interface, never the representation.

> 📖 **The Law of Leaky Abstractions (Joel Spolsky, 2002)** — *every abstraction leaks something — the skill is leaking only the interface, never the internals.*

Getters are asking

Tell, Don't Ask

Getters and setters are how we ask an object for its data. Tell, Don't Ask: put the behavior where the data lives, so the object acts on itself — fewer accessors, more functional cohesion.

Ask — getters pull data out

```
if (account.getBalance() >= amount) {
  account.setBalance(
     account.getBalance() - amount);
}
// behavior lives outside Account
```

Tell — the object acts on itself

```
account.withdraw(amount);
class Account {
  private long balance;
  void withdraw(long amount) {
    if (balance < amount) reject();
    balance -= amount;
  }
}
```

The "bad trick": make a field private, then add a public getter and setter.

That is fake encapsulation — the field is still exposed, just through a slower door. Hide the data, expose **behavior**.

The golden rule

High Cohesion, Low Coupling

Group what changes together. Separate what changes apart.

High cohesion pulls together

- Related behavior lives in one place
- Easy to find and understand

Low coupling keeps apart

- Unrelated parts stay independent
- Easy to change one without the others

Section 03

Designing for Change

Cohesion and coupling only matter because software changes.

The real goal

We Optimize for Change

You will never predict every future requirement. So you don't design to predict change — you design to absorb it cheaply when it comes.

- Isolate what varies — put likely changes behind a boundary.
- Hide decisions — callers shouldn't depend on how something works.
- Depend on abstractions, not details (we'll formalize this in SOLID).
- Small, cohesive, loosely coupled parts = a system that bends, not breaks.

The main tool

Encapsulation & Information Hiding

Encapsulation is how we buy low coupling: expose a small, stable surface; hide the volatile details behind it.

Expose (stable)

- What the module does
- A small, intention-revealing API
- Types the caller truly needs

Hide (volatile)

- How it does it — algorithms, data
- Internal fields & helper types
- Third-party libraries & storage

How internals escape

Breaking Information Hiding

Encapsulation hides the representation behind a stable interface. Here is how it leaks back out — getters and setters are only the most common way:

- **Public fields**
  The representation IS the interface.
- **Leaking implementation types**
  Return ArrayList/HashMap, not a role.
- **Getters & setters**
  A field exposed through a slower door.
- **Callers must know the steps**
  init() before use(), or magic flags.
- **Returning internal state**
  Hand back the live list; callers mutate it.
- **protected fields**
  Internals handed to every subclass.

The fix is always the same — **expose behavior, hide the representation.**

Section 04

The Law of Demeter

A simple rule to stop coupling from creeping back in.

The principle

Don't Talk to Strangers

The Law of Demeter (the 'principle of least knowledge') says a method should only talk to its immediate friends — not reach through them to strangers.

- Only call methods on: itself, its fields, its parameters, objects it creates.
- Avoid long chains: a.getB().getC().doSomething() reaches through others.
- Each dot past the first is a new thing you've secretly coupled to.

The smell

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.demeter`

The 'Train Wreck'

*This line knows the shape of Order, Customer and Address — **three types**. Change any one and it may break:*

Java

```
String city = order
  .getCustomer()
  .getAddress()
  .getCity();
// reaches 3 classes
```

C#

```
var city = order
  .Customer
  .Address
  .City;
// reaches 3 classes
```

Python

```
city = (order
  .customer
  .address
  .city)
# reaches 3 classes
```

Every getter in the chain is a dependency in disguise.

The fix

▶ runnable dev.kaldiroglu.bootcamp.fundamentals.demeter

Tell, Don't Ask

Ask the object you know to do the job — let it deal with its own internals:

C#

The chain is gone — Order hides its structure

```
// Instead of reaching through order for the city...
var city = order.ShippingCity();   // ask the object you have

class Order {
    public string ShippingCity() {
        return customer.ShippingCity();   // each object knows its own parts
    }
}
```

Now callers depend only on Order. Its internal shape is free to change.

Balance

Powerful — but Not Absolute

It buys you

- Lower coupling between classes
- Freedom to change internals
- Code that reads as intent, not plumbing

Keep perspective

- It's a heuristic, not a hard law
- Fluent builders & streams are fine
- Aim for less knowledge, not zero dots

Fine — not train wrecks:

```
list.stream().filter(x).map(y).toList()   ·   new Url().host(h).port(p).build()
```

Every dot returns the same object — you never reach through to a new stranger.

Run it yourself

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 13 tests here

`fundamentals/cohesion/`
UtilsSmell vs three focused classes

`fundamentals/coupling/`
structured ladder + OOP coupling & DIP

`fundamentals/password/`
a cohesive value object and its port

C# / .NET

xUnit · 9 tests here

`Fundamentals.cs`
cohesion, coupling, Demeter, leaks

`Password.cs`
the Password value object + hasher port

`…Tests.cs`
a fake DB and a recording mailer

Python

pytest · 9 tests here

`fundamentals.py`
cohesion, coupling, Demeter, leaks

`password.py`
value object + PasswordHasher protocol

`tests/`
test_fundamentals · test_password

`github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · -DOTNET · -PYTHON`

Every smell keeps its fix beside it. The tests assert they behave the same — so the difference you are being shown is structure, not behavior.

Clone one, run the suite, then break something and watch which test goes red. ***Across all chapters the suites are 70 · 63 · 65 tests.***

Remember these

Key Takeaways

- ✓ Cohesion (high) and coupling (low) are the two forces of good design.
- ✓ Cohesion: one module, one job. Coupling: know as little as possible.
- ✓ Coupling is a change multiplier — it turns small edits into big ones.
- ✓ Encapsulation buys low coupling: expose a stable surface, hide the details.
- ✓ Law of Demeter: talk to friends, not strangers — kill the train wrecks.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Structured Design** Yourdon & Constantine · Prentice Hall, 1979
Where cohesion and coupling were first defined — the original ladder we teach.

📖 **Code Complete, 2nd ed.** Steve McConnell · Microsoft Press, 2004
Ch. 5–7 on design: complexity, cohesion, coupling — evidence-based and language-agnostic.

📖 **Practical Object-Oriented Design, 2nd ed.** Sandi Metz · Addison-Wesley, 2018
Dependencies and messages, taught as concrete moves rather than principles.

📖 **Object-Oriented Software Construction, 2nd ed.** Bertrand Meyer · Prentice Hall, 1997
Design by Contract, and the strictest treatment of module design ever written.

📖 **Object-Oriented Software Design in C++** Ronald Mak · Manning, 2024
Bad design beside good, chapter after chapter — this course's method, book-length.

📖 **Software Design for Python Programmers** Ronald Mak · Manning, 2026
The same principles and patterns, in the third language this course uses.

Law of Demeter — the original OOPSLA '88 paper → `www2.ccs.neu.edu/research/demeter`

Next session

What's Next

Module 3

Clean Code & SOLID

with Akın Kaldıroğlu

Turning these forces into everyday habits: a Clean Code framework, then the five SOLID principles that make high cohesion and low coupling systematic.

Further practice

Find These Smells for Real

Every force from this session lives in the ACME Orders codebase. Measure the cohesion and coupling, then improve them.

Refactoring course · Cohesion & Coupling

Repo `github.com/kaldiroglu/refactoring-course-student-materials`

IN THIS SESSION, LOOK AT

- `Exercises/Ex01-CohesionCoupling.md`
- `.../service/OrderService.java` (coupled god-service)
- `.../util/Util.java` (low cohesion)

Questions?

High cohesion, low coupling — everything else is detail.